

Tp 4 - Variables

1. HECHO EN HOJA APARTE

2.

- a. La inicialización de variables puede clasificarse en estática y dinámica. La inicialización estática es aquella que se realiza antes de la ejecución del programa, durante la compilación. La inicialización dinámica puede ser implícita, cuando el lenguaje asigna un valor por defecto, o explícita, cuando el programador asigna un valor. Además puede ser diferida si la asignación ocurre en un momento posterior a la declaración. Formas de inicializar una variable:

1. Inicialización estática: se realiza antes de la ejecución del programa. Típico en variables globales.
`int x = 10 // global (ya esta cargado antes de ejecutar)`
2. Inicialización dinámica implícita: el lenguaje asigna automáticamente un valor por defecto. Ej: en Java → 0, false, null. No lo escribe el programador.
3. Inicialización dinámica diferida: la variable se declara primero y se inicializa después.
`int x;
x = 10;`
4. Inicialización dinámica explícita: ocurre durante la ejecución del programa. Se ejecuta como una instrucción mas. La hace el programador mediante una asignación. Por ej., en C → `int x = 10`

b.

Lenguaje	Inicialización estática	Inicialización dinámica implícita	Inicialización dinámica explícita	Inicialización diferida
Java	Si (atributos estáticos y globales)	Si (valores por defecto: 0, null, false)	Si (<code>int x = 10; </code>)	Si
C	Si (variables globales)	Parcial (globales si; locales no: basura)	Si (<code>int x = 10; </code>)	Si

Python	No explicita	No (no hay valores por defecto automáticos)	Si (x = 10)	No (no se puede usar sin asignar)
Ruby	No explicita	Si (nill por defecto)	Si (x = 10)	No

3. a. Atributo l - valor (dirección en memoria, donde vive):

a. Variable estática: se crea antes de ejecutar el programa.

- Dirección de memoria fija
- No cambia
- Existe durante todo el programa
- Nunca se destruye hasta que termina

Ejemplo en C: static int x;

b. Variable automática o local: Se crea al entrar en el bloque y se destruye al salir (ej: una función).

Ejemplo en C: void foo() {
 int x = 10;
 }

c. Variable dinámica: se crea en tiempo de ejecución manualmente.

- Se crea con new / malloc
- El programador controla la creacion y destruccion

Ejemplo en Java: Integer x = new Integer(5);

d. Variable semidinamica: se crea en ejecución como la dinámica PERO su tamaño puede depender de datos en la runtime

- No lo controla el programador
- El lenguaje maneja su vida automáticamente

Ejemplo en C: int n;
 scanf("%d", &n); // leo desde teclado
 int v[n]; // tamaño depende de n

En C

Tiene:

- Estáticas → globales, static
- Automáticas → variables locales
- Dinámicas → malloc, free

- Semidinámicas → VLA (int v[n])

En Ada

es mas formal:

- Estáticas → variables declaradas normalmente
- Dinámicas → usando access (punteros)
- Automáticas → variables en bloques/procedimientos

Ada controla mas la seguridad (menos basura que C).

4.

- a. Variable local: se declara dentro de un bloque de código (función, método o procedimiento).

- Solo existe dentro de ese bloque
- No se pueden usar fuera de la función donde fue declarada
- Se crea al entrar al bloque y se destruye al salir

Ejemplo: void function() { int x = 10; // variable local }

Variable global: se declara fuera de todas las funciones o bloques.

- Se puede usar desde cualquier parte del programa
- Existe durante toda la ejecución del programa
- Puede ser modificada por cualquier función (si lo permite el lenguaje)

Ejemplo: int x = 10; // variable global
void function() { x = x + 1; // acceso permitido }

- b. Si, una variable local puede ser estática respecto de su l-valor. Una variable local se crea al entrar a la función y se destruye al salir. Pero si la declaro como estática (static) cambia el comportamiento. Pasa a tener l-valor permanente (no se destruye al salir del bloque). Se llama variable local estática.

Ejemplo en C: int main() {
 static int contador = 0;
 contador++;
 printf(contador);
 return 0;
}

contador es local a main pero al ser static no se destruye al salir del main, mantiene su valor si se vuelve a ejecutar la función.

- c. Toda variable global tiene almacenamiento estático, independientemente de su tipo, ya que su tiempo de vida abarca toda la ejecución del programa.

- d. La diferencia es que en una variable estática el ℓ - valor permanece fijo durante toda la ejecución, pero su contenido puede modificarse; en cambio, una constante mantiene inmutable su valor, independientemente de su ubicación en memoria.

5.

- a. En Ada, la clasificación entre constantes numéricas y comunes se debe a la forma en que se gestiona su tipo, su precisión y el momento de su evaluación por parte del compilador. Esta distinción maximiza la seguridad tipográfica y mejora la eficiencia del código:
- Constantes numéricas (números con nombre): valores literales sin especificar un tipo explícito, utilizando una expresión estática numérica. Se evalúan en tiempo de compilación. Ejemplo:
Max: constant := 100;
 - Constantes comunes (objetos constantes): son objetos cuyo valor no puede cambiar tras su inicialización, pero que sí tienen un tipo explícito definido. Se utilizan cuando la constante debe pertenecer a un tipo de dato específico para cumplir con las reglas estrictas de tipado de Ada. Se evalúa durante la elaboración del programa. Ejemplo:

X: constant Integer := 10;

b. $H \rightarrow$ compilación

$I \rightarrow$ compilación

$K \rightarrow$ compilación. Porque H e I ya son conocidas, entonces $H * I$ también.

6. El cambio de una variable global a una variable local static en C, el comportamiento en cuanto a asignación de memoria es idéntico, pero el alcance cambia.
- Ubicación en memoria: ambos utilizan asignación estática. Se almacenan en el segmento de datos del programa. No se crean en el stack ni desaparecen al terminar la función; persisten durante toda la ejecución
 - Comportamiento del valor: en ambos, x mantiene su valor entre llamadas a func1
 - Lo que cambia:
 - Caso original: func2 puede ver y modificar a x porque es global
 - Caso modificado: al moverla dentro de func1 como static, x se vuelve invisible para func2. Si func2 intentaría usar x , el código no compilaría.

Conclusión: en cuanto a cómo y dónde se reserva el espacio en RAM, es lo mismo. Pero funcionalmente, se rompe la visibilidad de la variable para el resto del archivo.

7.

Clase Persona

- id, nombreApellido, domicilio, dni, fechaNac: son variables instancia (locales a la clase). Su ciclo de vida depende del objeto.
- cantTotalPersona: es una variable de clase (global al contexto de la clase). Al ser static, existe una sola copia para todas las instancias.
- edad, fN: son variables locales (al método getEdad). Solo existen mientras se ejecuta el método y se almacenan en el stack.

Clase Domicilio

- id, calle, loc: son variables de instancia (atributos)
- nro: es una variable de clase (global al contexto de la clase) por ser static

8.

- Las tres variables (i, h, mipuntero) tienen el mismo **tiempo de vida** (línea 6 a 15) ya que están declaradas en el mismo bloque (programa principal). Sin embargo, la memoria dinámica apuntada por mipuntero existe solo entre new (línea 9) y dispose (línea 12)
- Las tres variables (i, h, mipuntero) tienen el mismo **alcance** (línea 6 a 15) ya que están declaradas en el mismo bloque (programa principal) y son visibles durante toda la ejecución del mismo.
- Al intentar escribir h no presenta error, ya que mipuntero^ representa un valor entero y puede sumarse con i.
- Al intentar asignarle a i la expresión h - mipuntero se produce un error, ya que no es posible realizar operaciones aritméticas entre una variable de tipo entero y un puntero. mipuntero representa una dirección de memoria, no un valor numérico.
- Si, existe otra entidad: la memoria dinámica apuntada por mipuntero. Su tiempo de vida comienza con la instrucción new(mipuntero) y finaliza con dispose(mipuntero). Su alcance no es léxico como el de las variables del programa, sino que está dado indirectamente por la variable puntero mipuntero, a través de la cual puede ser accedida mientras esta referencia sea válida.
- Las variables i, h y mipuntero son variables estáticas, ya que su l-valor se asigna en tiempo de compilación/entrada al bloque y no cambia durante la ejecución.

9.

- En C, cuando la variable deja de ser visible (sale del alcance) pero sigue existiendo en memoria (sigue viva):

```

#include <stdlib.h>

int* crear() {
    int* p = malloc(sizeof(int));
    *p = 10;
    return p;
}

int main() {
    int* x = crear();

    // aca ya no existe la variable local "p"
    // pero la memoria a la que apunta x sigue viva

    free(x);
}

```

- b. En Pascal, cuando la variable sigue siendo visible, pero ya no es valida en memoria:

```

program Pascal;
type tpuntero = ^integer;
var mipuntero: tpuntero;
begin
    new(mipuntero);
    mipuntero^:= 3;

    dispose(mipuntero);

    // mipuntero sigue existiendo (alcance activo) durante todo el bloque,
    // pero la memoria dinámica a la que apunta tiene un tiempo de vida
    // menor

end.

```

- c. En Ada, cuando la variable es visible en todo el bloque y es valida en memoria en todo el bloque:

```

procedure Ejemplo is
    X : constant Integer := 10;
begin
    Put_Line(Integer'Image(X));
end;

```

end Ejemplo;

10. En el caso de variables locales simples, el alcance y el tiempo de vida suelen coincidir. Sin embargo, esto no es una regla general, que en presencia de memoria dinámica, punteros o mecanismos de referencia, el tiempo de vida de ciertos valores puede extenderse mas allá del alcance de la variable o finalizar antes de que el bloque termine.

- En C:
 - Si se pasa la dirección de c (puntero) → otro código puede modificarlo indirectamente
 - Si se guarda su dirección en memoria dinámica:

```
int* p = &c; // dirección de variable  
return p; // retorna direcciones de variables locales. Lo que puede  
//provocar acceso a memoria invalida una vez finalizado  
// su alcance.
```

En variables locales simples, alcance y tiempo de vida coinciden. Pero en presencia de punteros esto puede dejar de cumplirse

- En Pascal:
 - en el caso de las variables locales simples, el alcance y el tiempo de vida coinciden.
Sin embargo, si:
 - Si se pasa por referencia (var en parámetros)
 - o se guarda su dirección (punteros)

es posible crear alias o referencias a la variable, lo que puede provocar que su valor sea accedido indirectamente.
normalmente coinciden pero pueden “escaparse referencias”
- En Ada:
 - es mas seguro:
 - no permite punteros
 - evita escapes peligrosos en variables normales

la coincidencia entre alcance y tiempo de vida es mas frecuente, ya que el lenguaje impone reglas estrictas sobre el manejo de memoria y el uso de referencias, evitando el uso libre de punteros y reduciendo la posibilidad de el acceso a datos fuera de su ámbito.

11.

a.

i. F

- ii. F
- iii. F
- iv. V

b. El tipo de dato es la definición del conjunto de valores posibles de una variable y las operaciones permitidas sobre ellos.

12.

Ident.	Tipo	r-valor	Alcance	Tiempo de vida
a (línea 4)	automática	basura	5-14	1-14
n (línea 4)	automática	basura	5-14	1-14
p (línea 4)	automática	basura	5-14	1-14
v1 (línea 5)	automática	basura	6-14	1-14
c1 (línea 6)	estática	10	7-14	1-14
v2 (línea 2)	semi dinámica	basura	3-7.6	7-7.6
c1 (línea 3)	automática	basura	4-7.6	7-7.6
c2 (línea 3)	automática	basura	4-7.6	7-7.6
p (inea 4)	automática	basura	5-7.6	7-7.6
q (línea 4)	automática	basura	5-7.6	7-7.6
new (línea 7.5.3)	dinámica	dirección de memoria	7.5.4	7.5.4 - 7.6

13. El nombre de una variable es solo un identificador sintáctico que permite referenciar en el programa, pero no influye en sus propiedades semánticas como el tipo de dato, el alcance, el tiempo de vida o el valor en memoria.

14.

Ident.	Tipo	r-valor	Alcance	Tiempo de vida
v1 (línea 1)				

15.

16. En JavaScript, los modificadores var, let, const y la ausencia de declaración afectan el alcance, la mutabilidad y la seguridad de las variables.

- var tiene alcance de función
- let y const tienen alcance de bloque
- la ausencia de declarador crea variables globales implícitas

En comparación, lenguajes como Ada fuerzan declaraciones explícitas y alcance de bloque, evitando variables globales implícitas y reduciendo errores de manejo de memoria y visibilidad.